11. pandas

Julia Gallucci HB.Sc, M.Sc, Ph.D.

Campbell Family Mental Health Research Institute, CAMH

Institute of Medical Science | University of Toronto

Learning objectives

- Understand what pandas is and why it is used.
- Create and manipulate Series and DataFrames.
- Import/export data from CSV files.
- Explore, subset, and filter data.
- Handle missing values.
- Perform basic aggregation and group operations.
- Apply transformations to data.

What is Pandas?

- Python library for data analyses and manipulation
- Works with tabular data (like Excel, or CSV files)
- Core data structures include:
  - DataFrames (2D labeled tables)
  - Series – 1D labeled array (single column)
- We usually import pandas with an alias:

```
import pandas as pd
```

	X	Y	Z
0			
1			
2			
3			
4			

DataFrames

	X	Y	Z
0	True	3.14	“o”
1	False	7.65	“a-”
2	True	2.56	“b+”
3	True	7.89	“b-”
4	False	1.23	“o”

- Columns are labeled with their names
- Rows also have label, or index
  - If the row label is not specified, pandas uses numbers by default
- Each column is a Series, or a 1D array, where values share a data type
- Unlike Numpy arrays, DataFrames can have columns with different data types
- **DataFrames are mutable**

DataFrames

	X	Y	Z
0	True	3.14	“o”
1	False	7.65	“a-”
2	True	2.56	“b+”
3	True	7.89	“b-”
4	False	1.23	“o”

- We can manually create a DataFrame using DataFrame()
  - **If a dictionary is passed into this function, the keys become the column names, and the values become the rows**

```
pd.DataFrame({
    "X": [True, False, True, True, False],
    "Y": [3.14, 7.65, 2.56, 7.89, 1.23],
    "Z": ["o", "a-", "b+", "b-", "o"]
})
```

DataFrames

- Of course, we are more likely to load data into a DataFrame than create one manually.
- Pandas has a read function for different file formats. For example, to read in a csv or other delimited file, we use **`pd.read_csv()`**, then pass in the local file path of the csv to read

```
df =
pd.read_csv('01_material/data/bicycle_thefts_4326.csv')
```

Inspecting DataFrames

- Once we got our data, we can start to understand it by checking attributes:
  - `dtypes` which give us column data types
  - `shape` which give us the row by column dimensions.

```
df.shape # output (25569, 33) meaning 25569 rows, 33 cols
```

```
df.dtypes
#the object dtype is python’s way of saying string and is
a catch-all for columns that don’t fit neatly into a more
specific type like int, float etc
```

Inspecting DataFrames

- We can also have a look at the first few rows using `head()` method
- Similarly, the last few rows with the `tail()` method

```
df.head() # 5 is the default argument, but we can also
specify the number of rows we want to return by passing it
as an integer
```

```
df.tail(3) # last 3
```

Renaming Columns in a DataFrames

- We can rename any columns in our DataFrame that may not be so intuitive
- We use the `rename()` method to do this. This typically takes a dictionary with the current column name as the key, and the new name as the value

```
df = df.rename(columns = {"cost_of_bike": "bike_cost" })
```

Profiling Columns in a DataFrames

- It can be useful to focus on a subset of columns and understand their values
- To select a single column, we can simply supply the name of the column in the [ ]
- Pandas returns the column as a Series
- To get unique values, we use the `unique()` series method

```
df["Status"].unique()
#Output
array(['STOLEN', 'RECOVERED', 'UNKNOWN'], dtype=object)
```

Profiling Columns in a DataFrames

- If we wanted to count how many times each value appears, we use the `value_counts()` method

```
df["Status"].value_counts()
#Output
STOLEN        24807
UNKNOWN         454
RECOVERED       308
Name: Status, dtype: int64
```

Profiling Columns in a DataFrames

- We can get an overview of the DataFrame by profiling it with the `info()` method
  - This gives us the shape, as the number of rows and columns
  - The column names and their dtype
  - The number of non-null values in each column, etc.

```
df.info()
#Output
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25569 entries, 0 to 25568
Data columns (total 33 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   _id                     25569 non-null  int64
```

Changing data types

- Before exploring our data frame any further, we can fix the occurrence date column because it's supposed to represent date and times
- To do so, we can use pandas `to_datetime()` function, passing the column to convert it and then reassign the output back to the column we are converting

```
# Convert occurrence date columns to datetime
df["Occurrence_Date"] =
pd.to_datetime(df["Occurrence_Date"])
```

All other data type conversions can be done with the `astype()` method!

```
# Convert Status columns to category
df["Status"] = df["Status"].astype("category")
```

Describing DataFrame

- To get a sense of the values in a DataFrame, we can use the `describe()` method
- This summarizes only numeric columns by default, passing `include= all` argument produces summary statistics for all the columns

```
df.describe(include = "all")
# Output includes counts, unique values, top values,
frequencies etc.
```

Dropping columns, creating new ones

- We can drop a column using the `drop()` method and passing the column name we want to drop in the columns keyword argument

```
df = df.drop(columns = "_id")
# Output would be the dataframe without _id column
```

In pandas, **most operations return a new DataFrame by default** and do **not modify the original DataFrame in place** unless you explicitly tell them to.
So, we need to assign this expression back to the variable to save our changes!

Dropping columns, creating new ones

- Adding a new column to a DataFrame looks similar to adding a key-value pair to a dictionary. At its simplest, we can assign a single value to repeat down a column

```
df["current_year"] = 2026
```

- You can also create a new column by extracting specific parts of an existing column.

```
df["Occurrence_Month"] = df["Occurrence_Month"].str[:3]
```

Filtering and selecting data

- Let's see if our DataFrame contains any NA values.
- We could identify them using `.loc[]`, which allows us to access rows and columns based on either a boolean array or row/column labels.
- In this case, the boolean array can be created using the `isna()` method, which returns True for missing values and False otherwise.

```
# access rows of data where line is NA
df.loc[df["Bike_Model"].isna()]
```

Filtering and selecting data

- Alternatively, we can use the DataFrame `query()` method, which takes a filter condition as a string, and returns a DataFrame of records that meet a condition

```
# access rows of data where line is NA
df.query("Bike_Model.isna()")
```

query() is slower than **loc[],** but it can be easier to read.

Filtering and selecting data

- We can use the `.loc[]` and `.isin()` to filter rows in a DataFrame by providing a list of values for a column

```
Models_to_keep = ["HYBRID","SIRRUS","ESCAPE"]
# Filter using .loc[]
df.loc[df["Bike_Model"].isin(Models_to_keep)]
```

`.isin(list)` checks if each element in the column exists in the list.
`.loc[]` selects the rows that meet the condition.

Replacing values with str.replace

- As we've seen when investigating string methods, we can replace values using the `str.replace()`

```
#This makes all the upper case A's lowercase
df["Bike_Model"] = (df["Bike_Model"].str.replace("A", "a"))
```

Grouping

Note, we can group by more than one column by passing a list into the `groupby()`

- A core workflow in pandas is split-apply-combine
  - Splitting data into groups
  - Applying a function to each group
  - Combining the result into a data structure
- This workflow starts by grouping data , calling the `groupby()` method

```
#Let's first group the data by bike model
Model_Groups = df.groupby("Bike_Model")
```

- Then, we can apply a specific function to each group, for example

```
#Identify unique manufacturers for each group
Model_Groups["Bike_Make"].unique()
```

Writing to file

- To save our modified DataFrame to a file for sharing or later use, we use the method `df.to_csv()`
- **First argument is the file path where the CSV will be saved.**
- **If no path is specified, it saves to the current working directory.**

```
df.to_csv("updated_df.csv")
```

Quiz: Pandas

1. What is the primary data structure in pandas?

A. List
B. Dictionary
C. Series and DataFrames
D. Arrays

Quiz: Pandas

2.Which of the following creates a pandas DataFrame from a dictionary?

A. pd.DataFrame([1,2,3])

B. pd.DataFrame({'A':[1,2],'B':[3,4]})

C. pd.Series({'A':1,'B':2})

D. DataFrame([{'A':1,'B':2}])

Quiz: Pandas

3.How do you select the first 5 rows of a DataFrame df?

A. df.head()

B. df.tail()

C. df.first()

D. None

Quiz: Pandas

4.How do you select the single column ‘age’ from the DataFrame df

A. df['age']
B. df$age
C. Both a & b
D. None

Quiz: Pandas

5.How do you drop rows with missing values from a DataFrame df?

A. df.dropna()
B. df.drop()
C. df.remove_na()
D. df.delna()

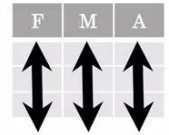
Exercise:

- Try This on Your Own
  - Read the CSV file `subway_stations.csv` into a DataFrame called df.
  - Display the first 5 rows of the DataFrame.
  - Display the last 2 rows of the DataFrame.
  - Check the number of rows and columns in the DataFrame.
  - Filter the DataFrame to show only rows where station is "COLLEGE STATION " or "ROYAL YORK STATION".
  - Export the filtered DataFrame to a CSV file called exported_data.csv.

Data Wrangling
with pandas
Cheat Sheet
http://pandas.pydata.org

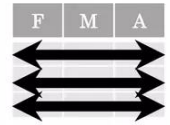
Tidy Data – A foundation for wrangling in pandas

In a tidy data set:



Each **variable** is saved in its own **column**

&



Each **observation** is saved in its own **row**

Tidy data complements pandas's **vectorized operations**. pandas will automatically preserve observations as you manipulate variables. No other format works as intuitively with pandas.

M * A

Syntax – Creating DataFrames

	a	b	c
1	4	7	10
2	5	8	11
3	6	9	12

```
df = pd.DataFrame(
        {"a" : [4 ,5, 6],
         "b" : [7, 8, 9],
         "c" : [10, 11, 12]},
       index = [1, 2, 3])
```

Specify values for each column.

```
df = pd.DataFrame(
    [[4, 7, 10],
     [5, 8, 11],
     [6, 9, 12]],
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

Specify values for each row.

		a	b	c
n	v			
d	1	4	7	10
	2	5	8	11
e	2	6	9	12

```
df = pd.DataFrame(
        {"a" : [4 ,5, 6],
         "b" : [7, 8, 9],
         "c" : [10, 11, 12]},
index = pd.MultiIndex.from_tuples(
        [('d',1),('d',2),('e',2)],
          names=['n','v']))
```

Create DataFrame with a MultiIndex

Method Chaining

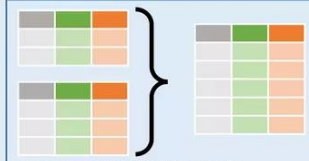
Most pandas methods return a DataFrame so that another pandas method can be applied to the result. This improves readability of code.

```
df = (pd.melt(df)
      .rename(columns={
              'variable' : 'var',
              'value' : 'val'})
      .query('val >= 200')
    )
```

Reshaping Data – Change the layout of a data set

pd.melt(df)
Gather columns into rows.

df.pivot(columns='var', values='val')
Spread rows into columns.



pd.concat([df1,df2])
Append rows of DataFrames

pd.concat([df1,df2], axis=1)
Append columns of DataFrames

df.sort_values('mpg')
Order rows by values of a column (low to high).

df.sort_values('mpg',ascending=False)
Order rows by values of a column (high to low).

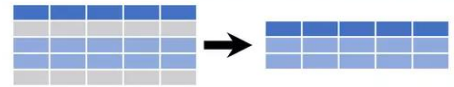
df.rename(columns = {'y':'year'})
Rename the columns of a DataFrame

df.sort_index()
Sort the index of a DataFrame

df.reset_index()
Reset index of DataFrame to row numbers, moving index to columns.

df.drop(columns=['Length','Height'])
Drop columns from DataFrame

Subset Observations (Rows)



df[df.Length > 7]
Extract rows that meet logical criteria.

df.drop_duplicates()
Remove duplicate rows (only considers columns).

df.head(n)
Select first n rows.

df.tail(n)
Select last n rows.

df.sample(frac=0.5)
Randomly select fraction of rows.

df.sample(n=10)
Randomly select n rows.

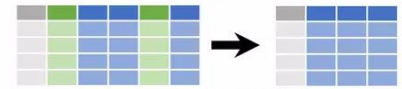
df.iloc[10:20]
Select rows by position.

df.nlargest(n, 'value')
Select and order top n entries.

df.nsmallest(n, 'value')
Select and order bottom n entries.

Logic in Python (and pandas)			
<	Less than	!=	Not equal to
>	Greater than	df.column.isin(*values*)	Group membership
==	Equals	pd.isnull(*obj*)	Is NaN
<=	Less than or equals	pd.notnull(*obj*)	Is not NaN
>=	Greater than or equals	&,|,~,^,df.any(),df.all()	Logical and, or, not, xor, any, all

Subset Variables (Columns)



df[['width','length','species']]
Select multiple columns with specific names.

df['width'] *or* **df.width**
Select single column with specific name.

df.filter(regex='*regex*')
Select columns whose name matches regular expression *regex*.

regex (Regular Expressions) Examples	
'\.'	Matches strings containing a period '.'
'Length$'	Matches strings ending with word 'Length'
'^Sepal'	Matches strings beginning with the word 'Sepal'
'^x[1-5]$'	Matches strings beginning with 'x' and ending with 1,2,3,4,5
'^(?!Species$).*'	Matches strings except the string 'Species'

df.loc[:,'x2':'x4']
Select all columns between x2 and x4 (inclusive).

df.iloc[:,[1,2,5]]
Select columns in positions 1, 2 and 5 (first column is 0).

df.loc[df['a'] > 10, ['a','c']]
Select rows meeting logical condition, and only the specific columns .